

Phase 7 : Orchestration et Automatisation (Apache Airflow)

1. Objectifs de l'Industrialisation

L' tape d'orchestration marque la transition entre un code de d veloppement ex cut  manuellement et une plateforme de donn es industrielle. Jusqu'  pr sent, les scripts d'ingestion, de nettoyage et de validation  taient d clench s   la demande. Pour garantir la fra cheur quotidienne des donn es et la fiabilit  du syst me sans intervention humaine, nous avons d ploy  Apache Airflow, accessible via <http://localhost:8090>. Ce choix technologique r pond au besoin de structurer les d pendances entre les t ches et de g rer de mani re autonome les reprises en cas d'incident technique.

2. Architecture du Pipeline (DAG)

Nous avons mod lis  le flux de traitement sous la forme d'un Graphe Orient  Acyclique (DAG) identifi  sous le nom `sirene_data_quality_pipeline`. Ce workflow est configur  via le param tre `schedule_interval='@daily'` pour s'ex cuter une fois par jour, assurant ainsi une mise   jour quotidienne des donn es.

Sur le plan technique, nous avons opt  pour l'utilisation du `BashOperator`. Ce choix permet d'ex cuter les scripts Python d velopp s lors des phases pr c dentes (`ingest_data.py`, `clean_data.py`, `validate_data.py`) comme des processus ind pendants et isol s. Cette approche modulaire facilite la maintenance : chaque script est une "bo te noire" qui peut  tre test e individuellement avant d' tre int gr e dans la cha ne d'orchestration.

La logique du pipeline est strictement s quentielle et se d compose en trois t ches interd pendantes. La premi re t che, `ingest_task`, se charge de r cup rer les donn es brutes. Une fois cette  tape valid e, la t che `clean_task` prend le relais pour effectuer les transformations et le nettoyage. Enfin, la t che `validate_task` lance la suite de tests `Great Expectations`. Les d pendances d finies (`>>`) garantissent que le nettoyage ne d marre jamais si l'ingestion a  chou , pr servant ainsi l'int grit  de l'entrep t de donn es.

3. R silience et Gestion des Erreurs

Pour pallier les  ventuelles instabilit s (micro-coupures r seau, indisponibilit  temporaire de la base de donn es), nous avons int gr  une politique de r silience directement dans la configuration du DAG. Les arguments par d faut (`default_args`) stipulent que chaque t che dispose d'un m canisme de reprise automatique (`retries: 1`) avec un d lai d'attente de 5 minutes (`retry_delay`). Concr tement, si un script  choue, Airflow attendra cinq minutes

avant de tenter une nouvelle exécution. Ce n'est qu'en cas de second échec que l'alerte définitive sera levée et le pipeline arrêté.

4. Code d'Orchestration Implémenté

Voici le code source du DAG déployé, illustrant l'enchaînement des tâches via les commandes Bash :

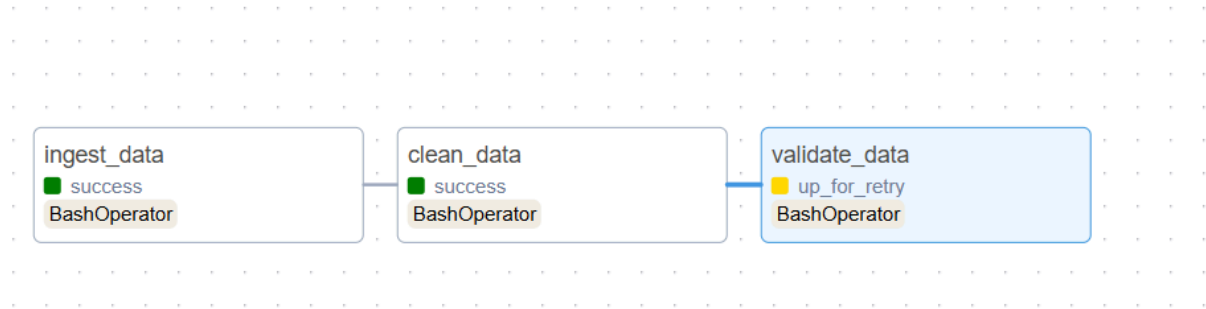
```

5  default_args: dict[str, Any] = {
6      'owner': 'data_engineer',
7      'retries': 1,
8      'retry_delay': timedelta(minutes=5),
9  }
10
11  with DAG(
12      dag_id='sirene_data_quality_pipeline',
13      default_args=default_args,
14      start_date=datetime(2023, 1, 1),
15      schedule_interval='@daily',
16      catchup=False
17  ) as dag:
18
19      # Task 1: Ingest CSV -> MariaDB Raw
20      ingest_task = BashOperator(
21          task_id='ingest_data',
22          bash_command='python /opt/airflow/scripts/ingest_data.py'
23      )
24
25      # Task 2: Clean Raw -> MariaDB Cleaned
26      clean_task = BashOperator(
27          task_id='clean_data',
28          bash_command='python /opt/airflow/scripts/clean_data.py'
29      )
30
31      # Task 3: Validate Cleaned Data (Great Expectations)
32      validate_task = BashOperator(
33          task_id='validate_data',
34          bash_command='python /opt/airflow/scripts/validate_data.py'
35      )
36
37      # Dependencies
38      ingest_task >> clean_task >> validate task

```

Capture d'écran de data_quality_pipeline.py

Au premier lancement du DAG, on avait un soucis avec la version de great_expectations :



On va donc forcer une version pour faire fonctionner correctement notre DAG :

```
docker-compose exec airflow-scheduler python -m pip install --user
"great_expectations==0.18.19" "sqlalchemy<2.0" psycopg2-binary
```

```
docker-compose exec airflow-scheduler python /opt/airflow/scripts/validate_data.py
```

Le "--user" débloque la situation.

